

Politechnika Gdańska
Wydział Elektrotechniki i Automatyki

Sprawozdanie z laboratorium

**Temat: Symulator Stanowiska
Dyspozytorskiego „Linii Produkcyjnej”**

Przedmiot: Informatyka i Systemy Operacyjne

Adam Błazejewski (198344)
Mateusz Kuczerowski (197900)

1. Sformułowanie zadania i przyjęte założenia

Celem zadania było zaprojektowanie i implementacja aplikacji okienkowej pełniącej funkcję symulatora stanowiska dyspozytorskiego nadzorującego proces pracy „linii produkcyjnej”. Program integruje elementy diagnostyki systemowej maszyny (parametry pracy komputera PC) oraz mechanizmy autodiagnostyki operatora mające na celu ciągłe badanie jego obecności i czujności (tzw. „czuwak”).

1.1. Założenia szczegółowe programu

- **Modelowanie procesu produkcyjnego:** Głównym kontrolowanym parametrem fizycznym jest temperatura silnika linii produkcyjnej. Temperatura ta zmienia się w czasie rzeczywistym w zależności od stanu pracy silnika (Engine WORKING/OFF) oraz systemu chłodzenia (Cooling Fan ON/OFF). Nominalna temperatura robocza została ustalona na poziomie 85.0°C. Do symulacji zachowania układu wykorzystano generator liczb losowych wprowadzający szum otoczenia.
- **Detekcja stanów awaryjnych:** System automatycznie wykrywa przekroczenia krytyczne parametrów termicznych. Przekroczenie 140.0°C skutkuje natychmiastowym zatrzymaniem awaryjnym (Machine Stop). Ponadto, praca w warunkach nieoptymalnych (poniżej 45.0°C lub powyżej 95.0°C) uruchamia licznik ostrzegawczy. Ignorowanie tego stanu przez 10 sekund również powoduje automatyczne zatrzymanie linii.
- **Układ autodiagnostyki operatora (Czuwak):** Program weryfikuje aktywność operatora za pomocą cyklicznego timera. Co 30 sekund operator ma obowiązek potwierdzić swoją przytomność poprzez wciśnięcie dedykowanego przycisku. Brak reakcji inicjuje 11-sekundowy stan ostrzegawczy (odliczanie krytyczne). Niepodjęcie działania w tym czasie wyzwala alarm dźwiękowo-wizualny i wyłącza proces w trybie bezpiecznym.
- **System poziomów uprawnień:** Zaimplementowano trójstopniową autoryzację użytkowników w oparciu o zewnętrzną bazę danych w formacie JSON (`users.json`):
 - *Viewer (Poziom 1):* Posiada wyłącznie uprawnienia do podglądu parametrów. Wszystkie elementy sterujące są zablokowane.
 - *Operator (Poziom 2):* Ma pełną kontrolę nad uruchamianiem i zatrzymywaniem silnika, resetowaniem alarmów oraz obsługą wentylatora chłodzącego.
 - *Admin (Poziom 3):* Oprócz uprawnień operatora posiada dostęp do modułów konfiguracyjnych pozwalających na dodawanie i modyfikację kont użytkowników.

2. Opis przyjętych rozwiązań programowych

Aplikacja została wykonana w środowisku **.NET Windows Forms** przy użyciu języka **C#**. Architektura opiera się na sterowaniu zdarzeniowym realizowanym za pomocą asynchronicznych komponentów `System.Windows.Forms.Timer`.

2.1. Algorytm symulacji termicznej silnika

Za obliczanie temperatury odpowiada metoda wywoływana przez `engineTempTimer` z interwałem 250 ms. Model uwzględnia bezwładność cieplną silnika oraz moc chłodniczą wentylatora. Poniższy kod (Listing 1) przedstawia kluczową logikę obliczeniową:

```
1  if (engineOn) {
2      if (fanState) {
3          double coolingPower = (engineTemperature / 30.0);
4          engineTemperature -= coolingPower + randomNoise;
5      } else {
6          if (engineTemperature < targetWorkTemp) {
7              double heatingPower = (targetWorkTemp - engineTemperature) /
20.0;
8              if (heatingPower < 0.2) heatingPower = 0.2;
9              engineTemperature += heatingPower + randomNoise;
10         } else {
11             double overheatingPower = (engineTemperature - targetWorkTemp)
/ 15.0;
12             if (overheatingPower < 0.2) overheatingPower = 0.2;
13             engineTemperature += overheatingPower + randomNoise;
14         }
15     }
16 }
```

Listing 1: Fragment metody `engineTempTimer_Tick` implementujący model termiczny

2.2. Obsługa procedury badania przytomności operatora

Układ „czuwaka” bazuje na osobnym zegarze `userAttentionTimer` (interwał 1 s). Odlicza on czas do konieczności interwencji ludzkiej. Wykazuje odporność na błędy i wymusza zatrzymanie maszyny w przypadku utraty łączności z operatorem:

```
1  private void userAttentionTimer_Tick(object sender, EventArgs e) {
2      if (engineOn == true) {
3          attentionCounter -= 1;
4          if (attentionCounter >= 0) {
5              timerLabel.Text = $"{attentionCounter}";
6          } else if (attentionCounter < 0) {
7              engineShutdownCounter -= 1;
8              timerLabel.Text = $"{engineShutdownCounter}";
9              userAttentionButton.Text = "USER INPUT NEEDED";
10             if (engineShutdownCounter <= 0) {
11                 machineStop();
12             }
13         }
14     }
15 }
```

Listing 2: Implementacja mechanizmu odliczania czasu czuwaka

2.3. Zarządzanie dostępem na podstawie ról

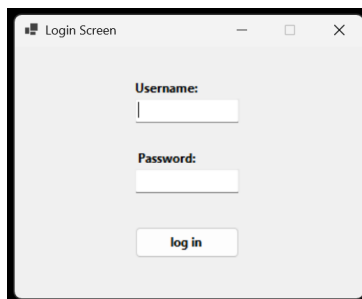
Weryfikacja ról odbywa się podczas inicjalizacji okna głównego (`mainForm_Load`). Pola przycisków administracyjnych oraz wykonawczych są aktywowane lub dezaktywowane na bazie zmiennej statycznej `confirmedUser.AuthorizationLvl`.

```
1 if (confirmedUser.AuthorizationLvl != 3) {  
2     addUserButton.Enabled = false;  
3     modifyUsersButton.Enabled = false;  
4 }  
5 if (confirmedUser.AuthorizationLvl == 1) {  
6     coolingButton.Enabled = false;  
7     engineStartButton.Enabled = false;  
8     engineStopButton.Enabled = false;  
9     eStopButton.Enabled = false;  
10    resetButton.Enabled = false;  
11 }
```

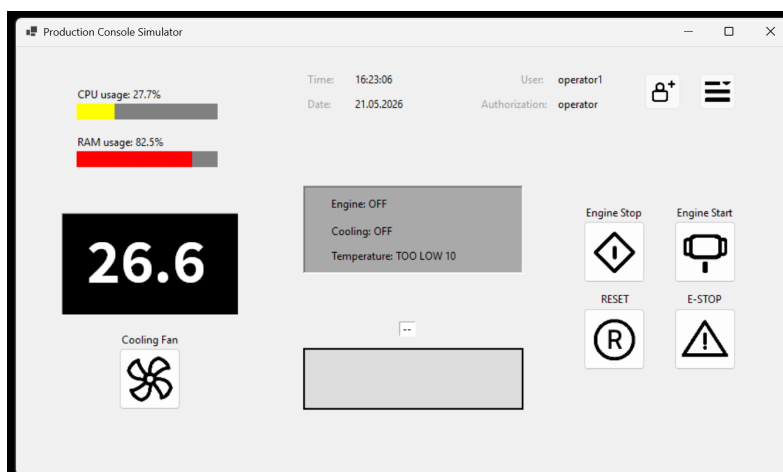
Listing 3: Logika blokowania kontrolek interfejsu GUI w zależności od poziomu autoryzacji

3. Prezentacja wyników i funkcjonalności

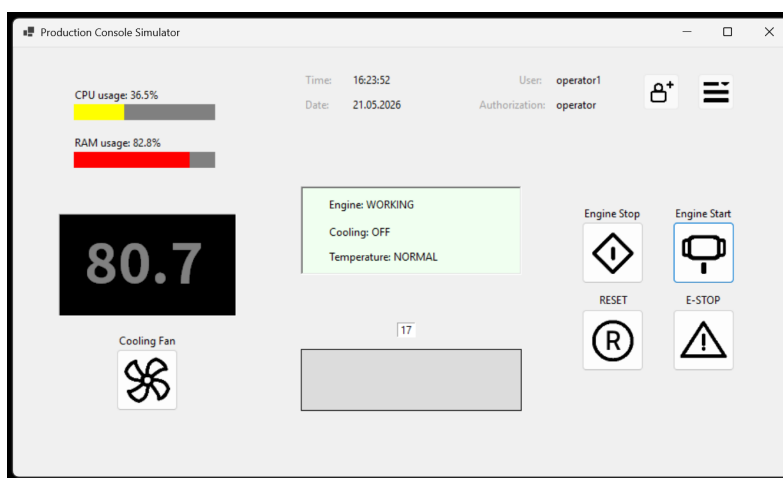
Poniżej przedstawiono poszczególne ekrany i funkcjonalności systemu zaimplementowane w aplikacji. Zgodnie z wytycznymi, zrzuty ekranu zostały zastąpione plikami zastępczymi.



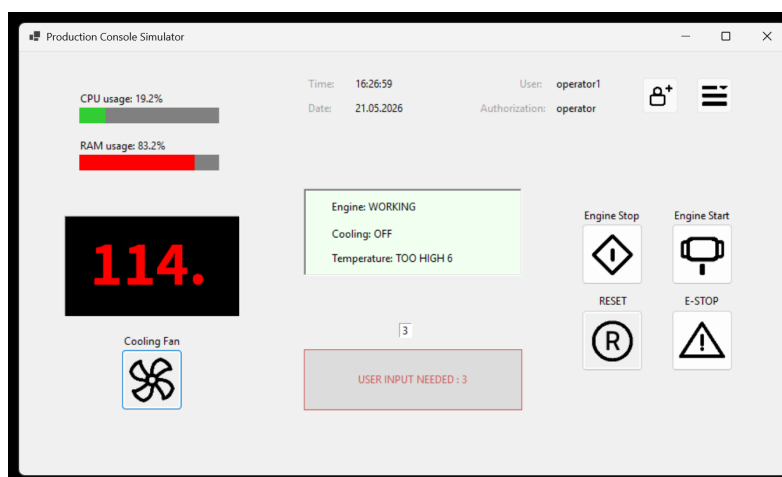
Rysunek 1: Okno logowania systemu (loginForm) pobierające dane uwierzytelniające i weryfikujące je z plikiem JSON.



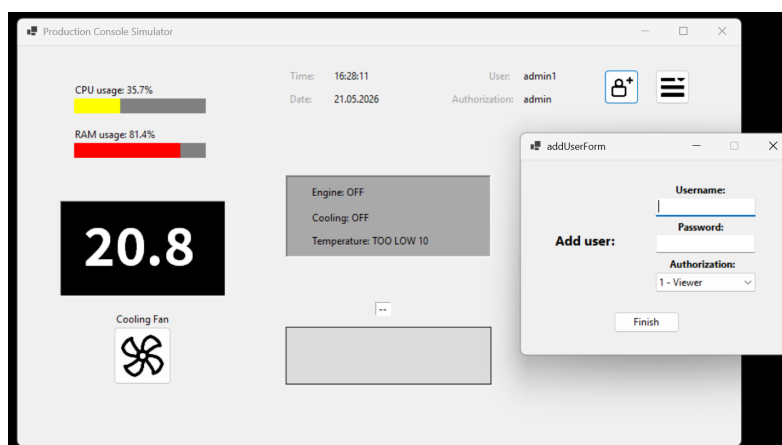
Rysunek 2: Główny pulpit sterowniczy w stanie spoczynku (Engine: OFF). Widoczne wskaźniki obciążenia procesora i pamięci RAM.



Rysunek 3: Praca linii produkcyjnej (Engine: WORKING). Temperatura silnika wzrasta dynamicznie w stronę punktu pracy.



Rysunek 4: Aktywacja alarmu autodiagnostycznego operatora. Przycisk centralny wymaga kliknięcia w celu zresetowania timera czuwaka.



Rysunek 5: Panel zarządzania bazą operatorów (editUsersForm), dostępny wyłącznie dla użytkowników z uprawnieniami administratora.

3.1. Zalety aplikacji

- **Bezpieczeństwo operacyjne:** Trójpoziomowy system ról całkowicie eliminuje ryzyko dokonania niepożądanych modyfikacji parametrów procesu przez osoby postronne (klasa Viewer) oraz zapewnia rozliczalność personelu.
- **Niezależny system autodiagnostyki:** Mechanizm aktywnego czuwaka z dwufazowym czasem ostrzegania skutecznie zabezpiecza proces przed zasłabnięciem lub nieuwagą operatora.
- **Czytelność interfejsu (HMI):** Kolorystyczna wizualizacja obciążenia CPU/RAM komputera PC (zielony, żółty, czerwony) pozwala na błyskawiczną ocenę stabilności platformy sprzętowej.

3.2. Wady i ograniczenia napisanego programu

- **Brak kryptografii haseł:** Dane uwierzytelniające użytkowników w pliku `users.json` są zapisywane czystym tekstem (plain text). W rzeczywistych systemach przemysłowych stanowi to krytyczną lukę bezpieczeństwa i wymaga zastosowania algorytmów haszujących (np. SHA-256 z solą).
- **Monolityczna architektura:** Logika symulacji fizycznej i obsługi zdarzeń została ściśle sprzężona z kodem formularza interfejsu graficznego (`mainForm.cs`). Utrudnia to rozwój aplikacji, skalowalność oraz wprowadzanie testów jednostkowych (brak separacji warstw według wzorca np. MVP lub MVVM).
- **Zależność platformowa:** Wykorzystanie biblioteki `LibreHardwareMonitor` oraz liczników `PerformanceCounter` ogranicza możliwość uruchomienia i pełnego działania aplikacji wyłącznie do systemów z rodziny Microsoft Windows.

4. Podsumowanie

Zaprojektowana i wykonana aplikacja w pełni realizuje wszystkie wymagania zawarte w sformułowaniu zadania projektowego. Symulator stanowiska dyspozytorskiego prawidłowo odzwierciedla dynamikę uproszczonego procesu produkcyjnego oraz skutecznie nadzoruje pracę i skupienie operatora. Wprowadzone wielopoziomowe zabezpieczenia (zarówno temperaturowe, jak i autoryzacyjne) dowodzą poprawności przyjętych koncepcji inżynierskich. Program stanowi solidną bazę wyjściową do budowy bardziej zaawansowanych systemów klasy SCADA.